

Android Platform

Android APIs with Jetpack Compose — a colorful pocket guide

📨 Intents
🔐 Permissions
🔔 Notifications
⏰ WorkManager
📡 Services
💾 Storage
📍 Location
📷 Camera
🏪 Google Play

Book 4 · 15 chapters · code + phone-screen previews · exercises & cheat sheet

Contents

01 🤖 The Android Platform 4

02 🌍 Resources & Localization 7

03 📨 Intents & Activity Results 10

04 🔐 Runtime Permissions 15

05 🔔 Notifications 19

06 ⏰ Background Work: WorkManager 23

07 📡 Services & Broadcasts 28

08 💾 Storage: DataStore & Files 33

09 📍 Location & Sensors 37

10 📷 Camera & Media 41

11 🔗 Connectivity & Deep Links 45

12 ♻️ Configuration, Process Death & Back 49

13 🐞 Debugging & Performance 53

14 🏪 Build & Publish 56

15 🚀 Mini Project: Water Reminder 60

★ ⚡ Cheat Sheet 66

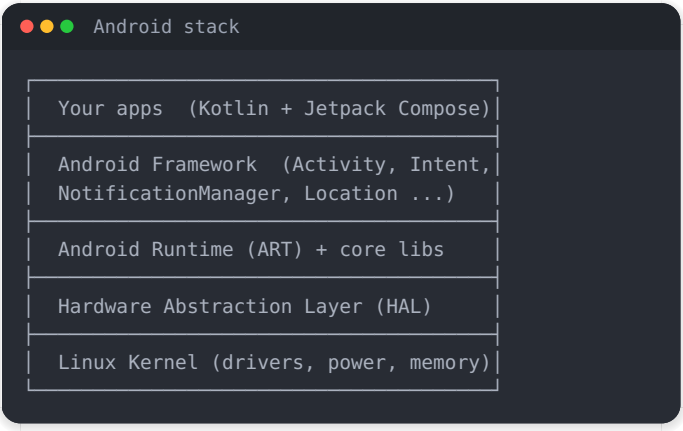
✓ ✅ Exercise Solutions 67

CHAPTER 1

The Android Platform

How Android works and the four app components

Before writing Android apps it helps to know what's underneath. Android is a full software stack built on Linux.



- Each app runs in its **own process** with its own Linux user — a **sandbox**.
- Kotlin code is compiled to **DEX** bytecode and run by **ART**.
- Android can **kill your process** any time it's in the background to free memory.

The four app components

Component	Purpose	Base class
🪟 Activity	A screen the user interacts with	ComponentActivity
⚙️ Service	Work without a UI (music, tracking)	Service
📡 Broadcast receiver	Reacts to system or app events	BroadcastReceiver
🗂️ Content provider	Shares data with other apps	ContentProvider

> 📌 **Remember**
> Every component must be declared in `AndroidManifest.xml`. Android starts components — you never call `MainActivity()` yourself.

The Application class

Created **once**, before any component. Ideal for app-wide setup.

MyApp.kt

```
class MyApp : Application() {
    override fun onCreate() {
        super.onCreate()
        // create notification channels, init
libraries...
        Log.d("MyApp", "App process started")
    }
}
```

AndroidManifest.xml

```
<application
    android:name=".MyApp"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name">
    ...
</application>
```

Context — your door to the system

A `Context` gives access to resources, files, system services and starting components.

Type	Lifetime	Use for
Application context	Lives as long as the app	Singletons, databases, WorkManager
Activity context	Lives with one screen	UI work, dialogs, starting activities

ContextDemo.kt

```
@Composable
fun ContextDemo() {
    val context = LocalContext.current          // in
Compose
    val appName = context.getString(R.string.app_name)
    val vibrator =
context.getSystemService(Vibrator::class.java)
    Text("Running $appName")
}
```

⚠️ **Watch out**

Never store an Activity context in a singleton or ViewModel — it causes **memory leaks**. Use `context.applicationContext` for long-lived objects.

CHAPTER 2

Resources & Localization



Strings, plurals, qualifiers and dark mode

Resources live in `res/`, separate from code. Android automatically picks the best version for the user's **language, screen and theme**.

res folder

```
res/
 ├ drawable/          ← vector icons, images
 ├ mipmap-*/          ← launcher icons
 ├ values/strings.xml ← default (English) texts
 ├ values-hy/strings.xml   ← Armenian 🇦🇲
 ├ values-ru/strings.xml   ← Russian
 ├ values-night/      ← dark-theme overrides
 ├ font/              ← custom fonts
 └ xml/               ← config files
```

Strings with arguments

values/strings.xml

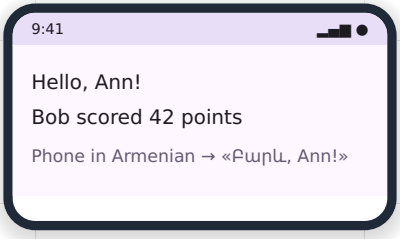
```
<resources>
    <string name="app_name">Water Buddy</string>
    <string name="welcome">Hello, %1$s!</string>
    <string name="score">%1$s scored %2$d points</
string>
</resources>
```

values-hy/strings.xml

```
<resources>
    <string name="app_name">Ջրի ընկեր</string>
    <string name="welcome">Բարև, %1$s!</string>
</resources>
```

Usage.kt

```
Text(stringResource(R.string.welcome, "Ann"))
Text(stringResource(R.string.score, "Bob", 42))
```



📱 What you see

Plurals

values/strings.xml

```
<plurals name="glasses">
    <item quantity="one">%d glass</item>
    <item quantity="other">%d glasses</item>
</plurals>
```

Plurals.kt

```
Text(pluralStringResource(R.plurals.glasses, count,
count))
// 1 → "1 glass",  5 → "5 glasses"
```

Common qualifiers

Folder	Used when
values-hy	Armenian language
values-night	Dark theme
values-w600dp	Width ≥ 600dp (tablets)
values-land	Landscape
drawable-xxhdpi	High-density screens
values-v31	Android 12+

Other resources in Compose

Resources.kt

```
Icon(painterResource(R.drawable.ic_water),
contentDescription = null)
val color = colorResource(R.color.brand)
val padding =
dimensionResource(R.dimen.screen_padding)
val isDark = isSystemInDarkTheme()
```

> 💡 **Tip**
> Never hard-code user-visible text. With **Translations Editor** (right-click strings.xml) you can add a language in minutes, and Android 13+ lets users pick a **per-app language**.

CHAPTER 3

Intents & Activity Results

Open other apps, share content and pick photos



An **Intent** is a message asking Android to do something: open a screen, a website, the dialer, or share text.

Explicit intent — open your own Activity

Explicit.kt

```kotlin
val intent = Intent(context,
SettingsActivity::class.java)
    .putExtra("from", "home")
context.startActivity(intent)

// in SettingsActivity:
val from = intent.getStringExtra("from")
```

📌 **Remember**

With Compose most apps have **one Activity** and use Navigation for screens. Explicit intents are mainly for system integration and multi-activity apps.

Implicit intents — let the system choose

Implicit.kt

```kotlin
// Open a website
context.startActivity(Intent(Intent.ACTION_VIEW,
    Uri.parse("https://kotlinlang.org")))

// Phone dialer (no permission needed)
context.startActivity(Intent(Intent.ACTION_DIAL,
    Uri.parse("tel:+37499123456")))

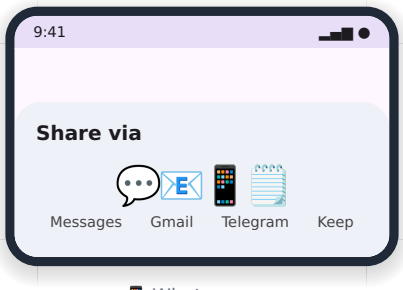
// Map
context.startActivity(Intent(Intent.ACTION_VIEW,
    Uri.parse("geo:40.1792,44.4991?q=Republic
Square")))

// Email
val email = Intent(Intent.ACTION_SENDTO).apply {
    data = Uri.parse("mailto:")
    putExtra(Intent.EXTRA_EMAIL,
arrayOf("hi@example.com"))
    putExtra(Intent.EXTRA_SUBJECT, "Feedback")
}
context.startActivity(email)
```

Share sheet

Share.kt

```kotlin
fun shareText(context: Context, text: String) {
    val send = Intent(Intent.ACTION_SEND).apply {
        type = "text/plain"
        putExtra(Intent.EXTRA_TEXT, text)
    }
    context.startActivity(Intent.createChooser(send,
"Share via"))
}
```



📱 What you see

> ⚠️ **Watch out**
> If no app can handle an implicit intent, `startActivity` throws `ActivityNotFoundException`. Wrap it in `try/catch` and show a friendly message.

Activity Result API — get something back

Use `rememberLauncherForActivityResult` with a ready-made **contract**.

PhotoPicker.kt

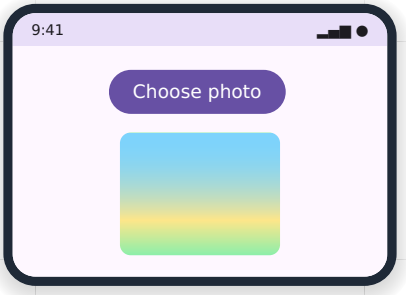
```
@Composable
fun PhotoPicker() {
    var photo by remember { mutableStateOf<Uri?
>(null) }

    val picker = rememberLauncherForActivityResult(
        ActivityResultContracts.PickVisualMedia()
    ) { uri -> photo = uri }        // null if
cancelled

    Column(horizontalAlignment =
Alignment.CenterHorizontally) {
        Button(onClick = {
            picker.launch(PickVisualMediaRequest(

ActivityResultContracts.PickVisualMedia.ImageOnly))
        }) { Text("Choose photo") }

        photo?.let {
            AsyncImage(model = it, contentDescription
= null,   // Coil
                modifier = Modifier.size(200.dp))
        }
    }
}
```



📱 What you see

📌 Remember

The **photo picker** needs **no storage permission**. The user picks only what they want to share.

Useful contracts

Contract	Does
PickVisualMedia()	Pick a photo/video
GetContent()	Pick any file by MIME type
TakePicture()	Take a photo into a Uri
CreateDocument(mime)	Let user choose where to save
OpenDocument()	Open an existing document
RequestPermission()	Ask for a permission
StartActivityForResult()	Any custom Activity

CHAPTER 4

Runtime Permissions

Asking the user for sensitive access



Android protects private data and hardware. **Normal** permissions (internet) are granted at install. **Dangerous** ones must be requested **at runtime**, and the user can refuse.

Permission	Protects
CAMERA	Camera
ACCESS_FINE_LOCATION	Precise location
RECORD_AUDIO	Microphone
POST_NOTIFICATIONS	Notifications (Android 13+)
READ_CONTACTS	Contacts
READ_MEDIA_IMAGES	Photos (prefer photo picker!)

1. Declare in the manifest

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera"
              android:required="false" />
```

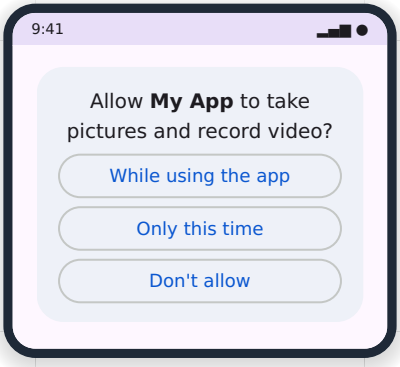
2. Check and request

CameraPermission.kt

```
@Composable
fun CameraFeature() {
    val context = LocalContext.current
    var granted by remember {
        mutableStateOf(
            ContextCompat.checkSelfPermission(
                context, Manifest.permission.CAMERA
            ) == PackageManager.PERMISSION_GRANTED
        )
    }
    val launcher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { isGranted -> granted = isGranted }

    if (granted) {
        Text("📷 Camera ready!")
    } else {
        Column {
            Text("We need the camera to scan QR
codes.")
            Button(onClick = {

launcher.launch(Manifest.permission.CAMERA)
            }) { Text("Allow camera") }
        }
    }
}
```



📱 What you see

Several at once

Multiple.kt

```
val launcher = rememberLauncherForActivityResult(

ActivityResultContracts.RequestMultiplePermissions()
) { results ->
    val fine =
results[Manifest.permission.ACCESS_FINE_LOCATION] ==
true
    val coarse =
results[Manifest.permission.ACCESS_COARSE_LOCATION]
== true
    // coarse only = user chose "Approximate"
}

launcher.launch(arrayOf(
    Manifest.permission.ACCESS_FINE_LOCATION,
    Manifest.permission.ACCESS_COARSE_LOCATION))
```

Best practices

📌 **Remember**

1. Ask **in context** — when the user taps the feature, not at app start.
2. Explain **why** before asking.
3. The app must still **work without** the permission.
4. Request only what you really need.

Permanently denied → open settings

After the user refuses twice, Android stops showing the dialog. Offer a button to the app's settings page:

OpenSettings.kt

```
fun openAppSettings(context: Context) {
    context.startActivity(

Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
              Uri.fromParts("package",
context.packageName, null))
    )
}
```

💡 **Tip**

Check the result after returning from Settings with `LifecycleResumeEffect` or by re-checking in `ON_RESUME`.

CHAPTER 5

Notifications

Channels, permission, taps and actions



1. Create a channel (once, at app start)

Since Android 8, every notification belongs to a **channel**. Users can mute channels separately.

MyApp.kt

```
const val CHANNEL_ID = "reminders"

class MyApp : Application() {
    override fun onCreate() {
        super.onCreate()
        val channel = NotificationChannel(
            CHANNEL_ID,
            "Reminders",                          //
shown in Settings
            NotificationManager.IMPORTANCE_DEFAULT
        ).apply { description = "Daily reminders" }


getSystemService(NotificationManager::class.java)
            .createNotificationChannel(channel)
    }
}
```

2. Permission (Android 13+)

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.POST_NOTIFICATIONS" /
>
```

AskNotifications.kt

```
val launcher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()) { }

LaunchedEffect(Unit) {
    if (Build.VERSION.SDK_INT >= 33) {

launcher.launch(Manifest.permission.POST_NOTIFICATIONS)
    }
}
```

3. Build and show

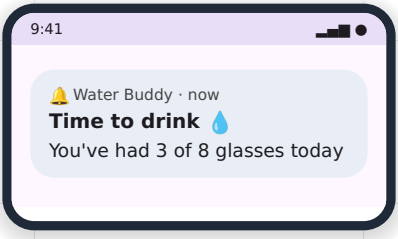
Notifier.kt

```
@SuppressLint("MissingPermission")
fun showReminder(context: Context) {
    val manager =
NotificationManagerCompat.from(context)
    if (!manager.areNotificationsEnabled()) return

    // open the app when tapped
    val openApp = PendingIntent.getActivity(
        context, 0,
        Intent(context, MainActivity::class.java),
        PendingIntent.FLAG_IMMUTABLE
    )

    val notification =
NotificationCompat.Builder(context, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_water)        //
white, simple icon
        .setContentTitle("Time to drink 💧")
        .setContentText("You've had 3 of 8 glasses
today")
        .setContentIntent(openApp)
        .setAutoCancel(true)                      //
dismiss on tap
        .build()

    manager.notify(1, notification)               //
id: 1
}
```

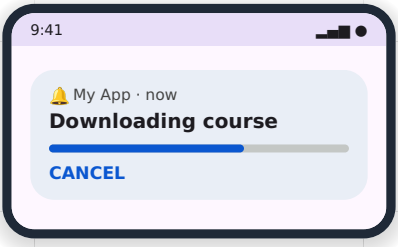


📱 What you see

Styles, actions & progress

Styles.kt

```
NotificationCompat.Builder(context, CHANNEL_ID)
    .setSmallIcon(R.drawable.ic_download)
    .setContentTitle("Downloading course")
    .setProgress(100, 65, false)            // max,
current, indeterminate
    .addAction(R.drawable.ic_close, "Cancel",
cancelPendingIntent)
    .setStyle(NotificationCompat.BigTextStyle()
        .bigText("Long text that expands when the user
pulls down..."))
    .setOngoing(true)
```



📱 What you see

Channel importance	Behavior
IMPORTANCE_HIGH	Sound + pops up on screen
IMPORTANCE_DEFAULT	Sound
IMPORTANCE_LOW	No sound
IMPORTANCE_MIN	Silent, collapsed

> ⚠️ **Watch out**
> Always use `PendingIntent.FLAG_IMMUTABLE` (or `FLAG_MUTABLE` only when really required). Android 12+ crashes without one of them.

CHAPTER 6

Background Work: WorkManager

Reliable tasks, even after reboot



WorkManager runs work that must finish **even if the app closes or the phone restarts**: syncing, uploading, cleanup, reminders.

Situation	Use
Right now, while app is open	Coroutines (viewModelScope)
Must finish, can wait a bit	WorkManager ✅
Repeating (≥ 15 min)	PeriodicWorkRequest ✅
User-visible & ongoing (music)	Foreground Service
At an exact time (alarm clock)	AlarmManager

1. Write a Worker

SyncWorker.kt

```
class SyncWorker(
    context: Context,
    params: WorkerParameters
) : CoroutineWorker(context, params) {

    override suspend fun doWork(): Result {
        val userId = inputData.getInt("userId", -1)
        return try {
            Api.syncNotes(userId)              //
suspend call
            Result.success(workDataOf("synced" to 12))
        } catch (e: IOException) {
            if (runAttemptCount < 3) Result.retry()
            else Result.failure()
        }
    }
}
```

2. Enqueue it

Schedule.kt

```kotlin
val constraints = Constraints.Builder()
    .setRequiredNetworkType(NetworkType.CONNECTED)
    .setRequiresBatteryNotLow(true)
    .build()

// one time
val once = OneTimeWorkRequestBuilder<SyncWorker>()
    .setConstraints(constraints)
    .setInputData(workDataOf("userId" to 42))
    .setBackoffCriteria(BackoffPolicy.EXPONENTIAL,
                        30, TimeUnit.SECONDS)
    .build()
WorkManager.getInstance(context).enqueue(once)

// repeating, only one copy ever
val daily = PeriodicWorkRequestBuilder<SyncWorker>(1,
TimeUnit.DAYS)
    .setConstraints(constraints)
    .build()
WorkManager.getInstance(context).enqueueUniquePeriodicWo
    "daily-sync", ExistingPeriodicWorkPolicy.KEEP,
daily)
```

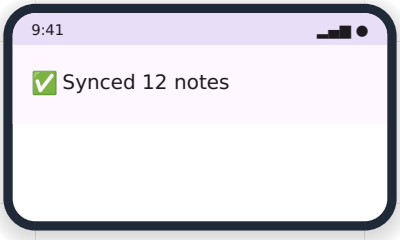
3. Observe progress in Compose

SyncStatus.kt

```
@Composable
fun SyncStatus(workId: UUID) {
    val context = LocalContext.current
    val info by remember(workId) {

WorkManager.getInstance(context).getWorkInfoByIdFlow(wor
    }.collectAsStateWithLifecycle(initialValue = null)

    Text(when (info?.state) {
        WorkInfo.State.ENQUEUED -> "⏳ Waiting for
network"
        WorkInfo.State.RUNNING -> "🔄 Syncing..."
        WorkInfo.State.SUCCEEDED -> "✅ Synced " +
            info?.outputData?.getInt("synced", 0) + "
notes"
        WorkInfo.State.FAILED -> "❌ Failed"
        else -> ""
    })
}
```



📱 What you see

> 💡 **Tip**
> Chain work: `WorkManager.getInstance(ctx).beginWith(compress).then(u`
> Cancel with `cancelUniqueWork("daily-sync")`.

⚠️ **Watch out**

Periodic work runs at most every **15 minutes** and timing is approximate — Android batches work to save battery.

CHAPTER 7

Services & Broadcasts

Foreground services and system events



Foreground service

For long, **user-visible** work like music playback, navigation or a workout timer. It must show a notification while running.

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.FOREGROUND_SERVICE" /
>
<uses-permission

android:name="android.permission.FOREGROUND_SERVICE_MEDI
>

<application ...>
    <service
        android:name=".MusicService"
        android:foregroundServiceType="mediaPlayback"
        android:exported="false" />
</application>
```

MusicService.kt

```kotlin
class MusicService : Service() {

    override fun onStartCommand(intent: Intent?,
flags: Int,
                                startId: Int): Int {
        val notification =
NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_music)
            .setContentTitle("Playing: Lo-fi beats
🎵")
            .setOngoing(true)
            .build()

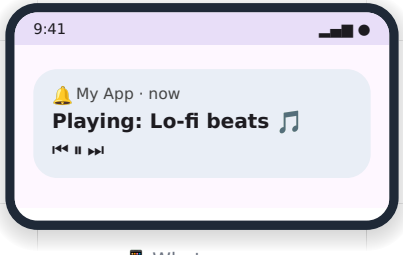
        ServiceCompat.startForeground(
            this, 1, notification,

ServiceInfo.FOREGROUND_SERVICE_TYPE_MEDIA_PLAYBACK)

        // start playback here...
        return START_STICKY
    }

    override fun onBind(intent: Intent?): IBinder? =
null

    override fun onDestroy() {
        // stop playback, release resources
        super.onDestroy()
    }
}
```



📱 What you see

Control.kt

```
// start
ContextCompat.startForegroundService(
    context, Intent(context,
MusicService::class.java))
// stop
context.stopService(Intent(context,
MusicService::class.java))
```

> 📌 **Remember**
> Since Android 14 every foreground service needs a **type** (mediaPlayback, location, camera, dataSync...) and the matching permission.

Broadcast receiver in Compose

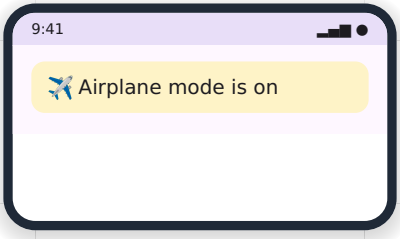
Listen to system events only while a screen is visible:

AirplaneBanner.kt

```
@Composable
fun AirplaneModeBanner() {
    val context = LocalContext.current
    var airplane by remember { mutableStateOf(false) }

    DisposableEffect(context) {
        val receiver = object : BroadcastReceiver() {
            override fun onReceive(c: Context,
intent: Intent) {
                airplane =
intent.getBooleanExtra("state", false)
            }
        }
        ContextCompat.registerReceiver(
            context, receiver,

IntentFilter(Intent.ACTION_AIRPLANE_MODE_CHANGED),
            ContextCompat.RECEIVER_NOT_EXPORTED)
        onDispose {
context.unregisterReceiver(receiver) }
    }
    if (airplane) Text("✈️ Airplane mode is on")
}
```



📱 What you see

Manifest receiver: after reboot

BootReceiver.kt

```
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent:
Intent) {
        if (intent.action ==
Intent.ACTION_BOOT_COMPLETED) {
            // reschedule alarms (WorkManager does
this itself)
        }
    }
}
```

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.RECEIVE_BOOT_COMPLETED"
>
<receiver android:name=".BootReceiver"
android:exported="true">
    <intent-filter>
        <action
android:name="android.intent.action.BOOT_COMPLETED" />
    </intent-filter>
</receiver>
```

⚠️ **Watch out**

`onReceive` runs on the main thread and must finish in about 10 seconds. For longer work, hand it to WorkManager.

CHAPTER 8

Storage: DataStore & Files

Settings, private files and user documents



What to store	Where
Small settings (theme, goal)	DataStore Preferences
Structured data, queries	Room (book 2)
Private files (cache, exports)	filesDir / cacheDir
Files the user owns	Storage Access Framework
Photos for the gallery	MediaStore

DataStore Preferences

SettingsRepository.kt

```
// top level, one instance per file
val Context.settings by preferencesDataStore(name =
"settings")

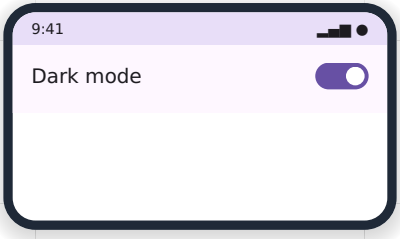
object Keys {
    val DARK = booleanPreferencesKey("dark_mode")
    val NAME = stringPreferencesKey("user_name")
}

class SettingsRepository(private val context:
Context) {
    val darkMode: Flow<Boolean> =
        context.settings.data.map { it[Keys.DARK] ?:
false }

    suspend fun setDarkMode(on: Boolean) {
        context.settings.edit { it[Keys.DARK] = on }
    }
}
```

DarkModeSetting.kt

```kotlin
@Composable
fun DarkModeSetting(repo: SettingsRepository) {
    val dark by
repo.darkMode.collectAsStateWithLifecycle(false)
    val scope = rememberCoroutineScope()
    Row(verticalAlignment =
Alignment.CenterVertically) {
        Text("Dark mode", Modifier.weight(1f))
        Switch(dark, onCheckedChange = {
            scope.launch { repo.setDarkMode(it) }
        })
    }
}
```



📱 What you see

> 📌 **Remember**
> DataStore is **asynchronous** (Flow + suspend) and safe from main-thread blocking. It replaces the old `SharedPreferences`.

Private app files

Files.kt

```
suspend fun saveNote(context: Context, text: String) =
    withContext(Dispatchers.IO) {
        File(context.filesDir,
"note.txt").writeText(text)
    }

suspend fun readNote(context: Context): String =
    withContext(Dispatchers.IO) {
        File(context.filesDir, "note.txt")
            .takeIf { it.exists() }?.readText() ?: ""
    }
```

Location	Behavior
filesDir	Private, kept until uninstall
cacheDir	Private, system may delete
getExternalFilesDir()	App-specific, visible via USB

Let the user choose where to save

Export.kt

```
@Composable
fun ExportButton(csv: String) {
    val context = LocalContext.current
    val saver = rememberLauncherForActivityResult(
        ActivityResultContracts.CreateDocument("text/
csv")
    ) { uri ->
        uri ?:
return@rememberLauncherForActivityResult

context.contentResolver.openOutputStream(uri)?.use {
            it.write(csv.toByteArray())
        }
    }
    Button(onClick = { saver.launch("report.csv") }) {
        Text("Export CSV")
    }
}
```

> 💡 **Tip**
> The Storage Access Framework needs **no permissions**: the system file picker grants access only to the file the user chose.

CHAPTER 9

Location & Sensors

Where the phone is and how it moves



Getting the current location

Add `play-services-location` and `kotlinx-coroutines-play-services` (for `.await()`).

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION"
>
<uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION" /
>
```

Location.kt

```
class LocationRepository(context: Context) {
    private val client =

LocationServices.getFusedLocationProviderClient(context)

    @SuppressLint("MissingPermission")    // checked
before calling
    suspend fun current(): Location? =
        client.getCurrentLocation(

Priority.PRIORITY_BALANCED_POWER_ACCURACY, null
        ).await()
}

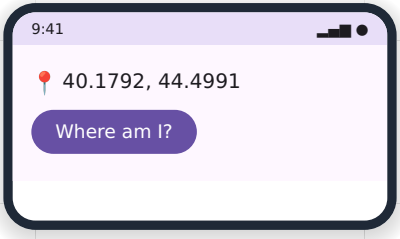
@Composable
fun WhereAmI(repo: LocationRepository) {
    var text by remember { mutableStateOf("Tap to
locate") }
    val scope = rememberCoroutineScope()
    val launcher = rememberLauncherForActivityResult(

ActivityResultContracts.RequestMultiplePermissions()
```

```
    ) { result ->
        if (result.values.any { it }) scope.launch {
            val loc = repo.current()
            text = loc?.let { "📍 %.4f, %.
4f".format(it.latitude,

it.longitude) }
                    ?: "Location unavailable"
        }
    }
    Column {
        Text(text)
        Button(onClick = { launcher.launch(arrayOf(
            Manifest.permission.ACCESS_FINE_LOCATION,

Manifest.permission.ACCESS_COARSE_LOCATION)) }) {
            Text("Where am I?")
        }
    }
}
```



📱 What you see

📌 Remember

Users can grant **approximate** location only. Make your feature work with coarse location when possible.

💡 Tip

Turn coordinates into an address with `Geocoder`, and show maps with the **Maps Compose** library: `GoogleMap { Marker(...) }`.

Sensors

Sensor	Measures
TYPE_ACCELEROMETER	Movement & tilt
TYPE_GYROSCOPE	Rotation speed
TYPE_LIGHT	Ambient light (lux)
TYPE_STEP_COUNTER	Steps since boot*
TYPE_PROXIMITY	Something near the screen

TiltMeter.kt

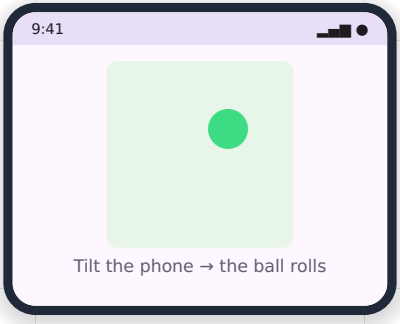
```
@Composable
fun TiltMeter() {
    val context = LocalContext.current
    var x by remember { mutableFloatStateOf(0f) }
    var y by remember { mutableFloatStateOf(0f) }

    DisposableEffect(Unit) {
        val sm =
context.getSystemService(SensorManager::class.java)
        val sensor =
sm.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
        val listener = object : SensorEventListener {
            override fun onSensorChanged(e:
SensorEvent) {
                x = e.values[0]; y = e.values[1]
            }
            override fun onAccuracyChanged(s:
Sensor?, a: Int) {}
        }
        sm.registerListener(listener, sensor,

SensorManager.SENSOR_DELAY_UI)
        onDispose { sm.unregisterListener(listener) }
    }


Box(Modifier.size(200.dp).background(Color(0xFFE8F5E9)))
{
        Box(Modifier
            .offset { IntOffset((80 - x *
```

```
10).dp.roundToPx(),
                                    (80 + y *
10).dp.roundToPx()) }
            .size(40.dp).clip(CircleShape)
            .background(Color(0xFF3DDC84)))
    }
}
```



📱 What you see

> ⚠️ **Watch out**
>
> * `TYPE_STEP_COUNTER` needs the `ACTIVITY_RECOGNITION` permission. Always unregister sensors — they drain the battery.

CHAPTER 10

Camera & Media

Take photos and use FileProvider



The simplest way to take a photo is to let the system camera app do it with the `TakePicture` contract. You give it a **Uri** to write into.

1. FileProvider — share a file safely

AndroidManifest.xml

```
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="$
{applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data

android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

res/xml/file_paths.xml

```
<paths>
    <cache-path name="photos" path="photos/" />
</paths>
```

2. Take the picture

CameraScreen.kt

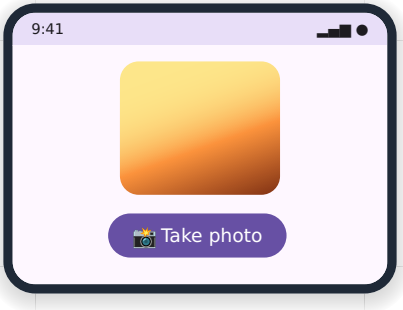
```
fun newPhotoUri(context: Context): Uri {
    val dir = File(context.cacheDir, "photos").apply
{ mkdirs() }
    val file = File(dir, "photo_$
{System.currentTimeMillis()}.jpg")
    return FileProvider.getUriForFile(
```

```
        context, "$
{context.packageName}.fileprovider", file)
}

@Composable
fun CameraScreen() {
    val context = LocalContext.current
    var pending by remember { mutableStateOf<Uri?
>(null) }
    var photo by remember { mutableStateOf<Uri?
>(null) }

    val camera = rememberLauncherForActivityResult(
        ActivityResultContracts.TakePicture()
    ) { success -> if (success) photo = pending }

    Column(horizontalAlignment =
Alignment.CenterHorizontally) {
        photo?.let {
            AsyncImage(it, "Photo",
Modifier.size(240.dp)
                .clip(RoundedCornerShape(16.dp)),
                contentScale = ContentScale.Crop)
        }
        Button(onClick = {
            pending = newPhotoUri(context)
            camera.launch(pending!!)
        }) { Text("📸 Take photo") }
    }
}
```



📱 What you see

> ⚠️ **Watch out**
> If your manifest declares the `CAMERA` permission, you must request it before launching `TakePicture`, otherwise it fails with a SecurityException.

Save into the gallery (MediaStore)

Gallery.kt

```
suspend fun saveToGallery(context: Context, bitmap:
Bitmap) =
    withContext(Dispatchers.IO) {
        val values = ContentValues().apply {
            put(MediaStore.Images.Media.DISPLAY_NAME,
"art_${System.currentTimeMillis()}.png")
            put(MediaStore.Images.Media.MIME_TYPE,
"image/png")

put(MediaStore.Images.Media.RELATIVE_PATH, "Pictures/
MyApp")
        }
        val resolver = context.contentResolver

resolver.insert(MediaStore.Images.Media.EXTERNAL_CONTENT
values)
            ?.let { uri ->
                resolver.openOutputStream(uri)?.use {

bitmap.compress(Bitmap.CompressFormat.PNG, 100, it)
                }
            }
    }
```

> 📌 **Remember**
> On Android 10+ saving your **own** images to MediaStore needs no storage permission.

> 💡 **Tip**
> Need a custom in-app camera (filters, QR scanning)? Use **CameraX** with `PreviewView` inside Compose's `AndroidView`, or the CameraX Compose viewfinder.

CHAPTER 11

Connectivity & Deep Links



Online status, and opening the app from links

Observe network status as a Flow

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" /
>
```

Connectivity.kt

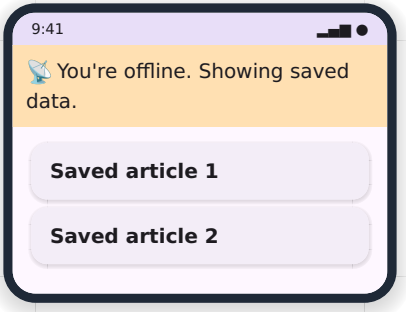
```
fun Context.isOnlineFlow(): Flow<Boolean> =
callbackFlow {
    val cm =
getSystemService(ConnectivityManager::class.java)
    val callback = object :
ConnectivityManager.NetworkCallback() {
        override fun onAvailable(network: Network) {
trySend(true) }
        override fun onLost(network: Network) {
trySend(false) }
    }
    trySend(cm.activeNetwork != null)        //
initial value
    cm.registerDefaultNetworkCallback(callback)
    awaitClose {
cm.unregisterNetworkCallback(callback) }
}.distinctUntilChanged()
```

OfflineBanner.kt

```kotlin
@Composable
fun OfflineBanner() {
    val context = LocalContext.current
    val online by remember { context.isOnlineFlow() }
        .collectAsStateWithLifecycle(initialValue =
true)

    AnimatedVisibility(!online) {
        Text("📡 You're offline. Showing saved data.",

Modifier.fillMaxWidth().background(Color(0xFFFFE0B2))
                .padding(8.dp))
    }
}
```



📱 What you see

> 📌 **Remember**
> `callbackFlow` is the standard way to turn any Android listener into a Kotlin Flow. `awaitClose` unregisters when collection stops.

Deep links

A deep link opens a specific screen from a URL: in a browser, email or notification.

AndroidManifest.xml

```
<activity android:name=".MainActivity"
android:exported="true">
    <intent-filter android:autoVerify="true">
        <action
android:name="android.intent.action.VIEW" />
        <category
android:name="android.intent.category.DEFAULT" />
        <category
android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="https"
              android:host="shop.example.com"
              android:pathPrefix="/product" />
    </intent-filter>
</activity>
```

AppNav.kt

```
@Serializable
data class Product(val id: Int)

NavHost(navController, startDestination = Home) {
    composable<Home> { HomeScreen() }
    composable<Product>(
        deepLinks = listOf(
            navDeepLink<Product>(
                basePath = "https://shop.example.com/
product")
        )
    ) { entry ->
        ProductScreen(entry.toRoute<Product>().id)
    }
}
// https://shop.example.com/product/42 →
ProductScreen(42)
```

> 💡 **Tip**
>
> Test a deep link from your computer: `adb shell am start -a android.intent.action.VIEW -d "https://shop.example.com/product/42"`

📌 **Remember**

`autoVerify` + a `assetlinks.json` file on your website make them **App Links**: they open your app directly, without the “Open with” dialog.

CHAPTER 12

Configuration, Process Death & Back



Survive everything Android throws at you

Your app must handle three surprises: **configuration changes** (rotation, language, dark mode), **process death** (Android kills your app in the background), and the **back gesture**.

Storage	Rotation	Process death
remember	❌	❌
rememberSaveable	✅	✅
ViewModel field	✅	❌
SavedStateHandle	✅	✅
Room / DataStore	✅	✅

SavedStateHandle in ViewModels

SavedState.kt

```
class SearchViewModel(
    private val savedState: SavedStateHandle
) : ViewModel() {

    // restored automatically after process death
    val query: StateFlow<String> =
        savedState.getStateFlow("query", "")

    fun onQueryChange(q: String) {
        savedState["query"] = q
    }
}

// Navigation arguments are in it too:
class DetailsViewModel(savedState:
SavedStateHandle) : ViewModel() {
    private val route = savedState.toRoute<Details>()
    val id = route.id
}
```

> 💡 **Tip**
>
> Test process death: open your app, press Home, then run `adb shell am kill com.example.myapp` and reopen it from Recents. Or enable **Don't keep activities** in Developer options.

Splash screen API

res/values/themes.xml

```
<style name="Theme.App.Starting"
parent="Theme.SplashScreen">
    <item name="windowSplashScreenBackground">@color/
brand</item>
    <item
name="windowSplashScreenAnimatedIcon">@drawable/
ic_logo</item>
    <item name="postSplashScreenTheme">@style/
Theme.App</item>
</style>
```

MainActivity.kt

```
class MainActivity : ComponentActivity() {
    private val vm: MainViewModel by viewModels()

    override fun onCreate(savedInstanceState:
Bundle?) {
        val splash = installSplashScreen()     //
before super!
        super.onCreate(savedInstanceState)
        splash.setKeepOnScreenCondition {
vm.isLoading.value }
        enableEdgeToEdge()
        setContent { AppTheme { AppNav() } }
    }
}
```



📱 What you see

> 📌 **Remember**
> Set `android:theme="@style/Theme.App.Starting"` on the activity in the manifest.

Back handling

Editor.kt

```
@Composable
fun EditorScreen(onExit: () -> Unit) {
    var text by rememberSaveable {
mutableStateOf("") }
    var askDiscard by remember {
mutableStateOf(false) }

    // intercept back only when there are unsaved
changes
    BackHandler(enabled = text.isNotEmpty()) {
askDiscard = true }

    TextField(text, { text = it },
Modifier.fillMaxSize())

    if (askDiscard) AlertDialog(
        onDismissRequest = { askDiscard = false },
        title = { Text("Discard changes?") },
        confirmButton = { TextButton(onClick =
onExit) { Text("Discard") } },
        dismissButton = {
            TextButton(onClick = { askDiscard =
false }) { Text("Keep") }
        })
}
```

> 💡 **Tip**
> Enable the **predictive back** animation with `android:enableOnBackInvokedCallback="true"` in `<application>` (it's the default when targeting newer Android versions).

CHAPTER 13

Debugging & Performance

Find bugs, crashes and slow code



Logcat like a pro

Logcat filter	Shows
tag:MainActivity	Only your tag
level:error	Errors only
package:mine	Only your app
is:crash	Crashes
message:timeout	Search message text

Reading a crash

Logcat

```
FATAL EXCEPTION: main
Process: com.example.myapp, PID: 8123
java.lang.NullPointerException: ...
    at
com.example.myapp.ProfileViewModel.load(ProfileViewMod
el.kt:42)
    at com.example.myapp.ProfileScreen...
```

📌 **Remember**

Read the **first line from your package**: here it's `ProfileViewModel.kt:42`. Click it in Android Studio to jump straight there.

The debugger

- Click the gutter to set a **breakpoint** 🔴, then run with **Debug** 🐞.
- **Step over** (F8), **step into** (F7), and inspect variables.
- **Evaluate expression** (Alt+F8) runs Kotlin code while paused.
- **Attach debugger** to an already-running app.

StrictMode — catch main-thread I/O

MyApp.kt

```
class MyApp : Application() {
    override fun onCreate() {
        super.onCreate()
        if (BuildConfig.DEBUG) {
            StrictMode.setThreadPolicy(
                StrictMode.ThreadPolicy.Builder()
                    .detectDiskReads()
                    .detectDiskWrites()
                    .detectNetwork()
                    .penaltyLog()
                    .build())
        }
    }
}
```

▶ **Logcat**

```
D/StrictMode: StrictMode policy violation;
~duration=48 ms:
  android.os.strictmode.DiskReadViolation

at ...SettingsRepository.load(SettingsRepository.kt:
18)
```

ANR — Application Not Responding

> ⚠️ **Watch out**
> If the main thread is blocked for about **5 seconds**, Android shows an ANR dialog. Causes: network or database on the main thread, long loops, deadlocks. Fix: move work to `Dispatchers.IO` / `Default`.

Tools overview

Tool	Use for
Android Profiler	CPU, memory, network, energy live
Layout Inspector	UI tree & Compose recompositions
App Inspection	Live Room DB, WorkManager, network
LeakCanary	Detects memory leaks automatically
Baseline Profiles	Faster startup & smoother scrolling
Firebase Crashlytics	Crash reports from real users

Useful adb commands

Terminal

```
adb devices                          # list connected
phones
adb install app-debug.apk            # install an APK
adb logcat *:E                       # errors only
adb shell pm clear com.example.app   # clear app data
adb shell am kill com.example.app    # simulate process
death
adb shell cmd uimode night yes       # force dark mode
```

CHAPTER 14

Build & Publish

Build types, flavors, signing and Google Play



Versions & build types

app/build.gradle.kts

```
android {
    defaultConfig {
        applicationId = "com.example.waterbuddy"
        versionCode = 3          // integer, +1 every
release
        versionName = "1.2.0"    // shown to users
    }

    buildTypes {
        debug {
            applicationIdSuffix = ".debug"   //
install side by side
        }
        release {
            isMinifyEnabled = true           // R8:
shrink + obfuscate
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-
android-optimize.txt"),
                "proguard-rules.pro")
        }
    }
    buildFeatures {
        compose = true
        buildConfig = true
    }
}
```

Product flavors

Flavors

```
android {
    flavorDimensions += "tier"
    productFlavors {
        create("free") {
            dimension = "tier"
            applicationIdSuffix = ".free"
            buildConfigField("Boolean", "SHOW_ADS",
"true")
        }
        create("pro") {
            dimension = "tier"
            buildConfigField("Boolean", "SHOW_ADS",
"false")
        }
    }
}

// in code:
if (BuildConfig.SHOW_ADS) AdBanner()
```

📌 **Remember**

Flavors × build types = **build variants**: freeDebug, freeRelease, proDebug, proRelease. Choose one in the *Build Variants* panel.

⚠️ **Watch out**

R8 can remove classes used only via reflection (e.g. some serialization or JSON models). Test the **release** build and add `-keep` rules when needed.

Signing & App Bundle

- **Build → Generate Signed App Bundle** creates an `.aab` file.

- Create an **upload key** (keystore). Back it up and keep the password safe.
- **Play App Signing**: Google keeps the real app signing key, so a lost upload key can be reset.

> ⚠️ **Watch out**
> Never commit keystores or passwords to Git. Store them in `local.properties` or CI secrets.

Publishing on Google Play

Step	What to do
1	Create a Google Play developer account
2	Create the app, fill store listing (icon, screenshots, description)
3	Complete policy forms: privacy policy, data safety, content rating
4	Upload the .aab to **Internal testing** first
5	Closed test with real testers (required for new personal accounts)
6	Promote to **Production**, optionally as a staged rollout

> 💡 **Tip**
> Google Play regularly raises the minimum `targetSdk`. Check the current requirement in the Play Console before each release.

Release checklist

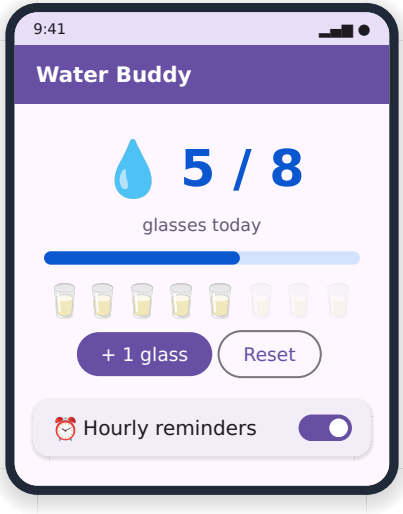
	Check
☐	versionCode increased
☐	Release build tested on a real phone
☐	No debug logs with private data
☐	Works offline & after process death
☐	All permissions really needed
☐	Dark mode, big fonts & RTL look fine
☐	Crash reporting enabled

CHAPTER 15

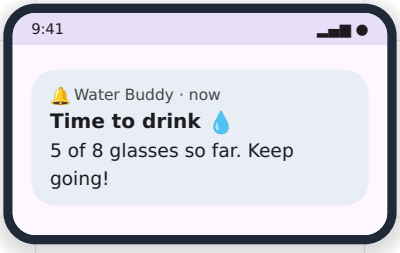
Mini Project: Water Reminder

Permissions + notifications + WorkManager + DataStore

A real Android app that uses platform features together: **DataStore** for counts, a **notification channel**, the **notification permission**, and **WorkManager** for hourly reminders, with a Compose UI.



What you see



What you see

Manifest

AndroidManifest.xml

```
<uses-permission
android:name="android.permission.POST_NOTIFICATIONS" /
>

<application
    android:name=".WaterApp"
    android:label="Water Buddy" ...>
    <activity android:name=".MainActivity"
android:exported="true">
        <!-- MAIN / LAUNCHER intent-filter -->
    </activity>
</application>
```

Data & background work

Water.kt

```
val Context.waterStore by
preferencesDataStore("water")
val GLASSES = intPreferencesKey("glasses")
val REMINDERS = booleanPreferencesKey("reminders")
const val GOAL = 8
const val CHANNEL_ID = "water"

class WaterApp : Application() {
    override fun onCreate() {
        super.onCreate()

getSystemService(NotificationManager::class.java)
```

```
            .createNotificationChannel(NotificationChann
                CHANNEL_ID, "Water reminders",

NotificationManager.IMPORTANCE_DEFAULT))
    }
}

class ReminderWorker(ctx: Context, params:
WorkerParameters)
    : CoroutineWorker(ctx, params) {

    @SuppressLint("MissingPermission")
    override suspend fun doWork(): Result {
        val ctx = applicationContext
        val nm = NotificationManagerCompat.from(ctx)
        if (!nm.areNotificationsEnabled()) return
Result.success()

        val glasses = ctx.waterStore.data.first()
[GLASSES] ?: 0
        if (glasses >= GOAL) return
Result.success()  // goal reached

        val open = PendingIntent.getActivity(ctx, 0,
            Intent(ctx, MainActivity::class.java),
            PendingIntent.FLAG_IMMUTABLE)

        nm.notify(1, NotificationCompat.Builder(ctx,
CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_water)
            .setContentTitle("Time to drink 💧")
            .setContentText("$glasses of
$GOAL glasses so far. Keep going!")
            .setContentIntent(open)
            .setAutoCancel(true)
            .build())
        return Result.success()
    }
}

fun setReminders(context: Context, on: Boolean) {
    val wm = WorkManager.getInstance(context)
    if (on) {
        wm.enqueueUniquePeriodicWork("water",
            ExistingPeriodicWorkPolicy.KEEP,
```

```
PeriodicWorkRequestBuilder<ReminderWorker>(
                1, TimeUnit.HOURS).build())
    } else {
        wm.cancelUniqueWork("water")
    }
}
```

UI

MainActivity.kt

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState:
Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent { MaterialTheme { WaterScreen() } }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun WaterScreen() {
    val context = LocalContext.current
    val scope = rememberCoroutineScope()
    val prefs by remember { context.waterStore.data }
        .collectAsStateWithLifecycle(initialValue =
null)
    val glasses = prefs?.get(GLASSES) ?: 0
    val reminders = prefs?.get(REMINDERS) ?: false

    fun saveReminders(on: Boolean) = scope.launch {
        context.waterStore.edit { it[REMINDERS] = on }
        setReminders(context, on)
    }

    val permission =
rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { granted -> if (granted) saveReminders(true) }

    Scaffold(topBar = {
        TopAppBar(title = { Text("Water Buddy") })
    }) { padding ->
        Column(
```

```
Modifier.padding(padding).padding(24.dp).fillMaxWidth(),
            horizontalAlignment =
Alignment.CenterHorizontally
        ) {
            Text("💧 $glasses / $GOAL",
                 style =
MaterialTheme.typography.displayMedium)
            Text(pluralStringResource(
                R.plurals.glasses_today, glasses,
glasses))
            Spacer(Modifier.height(16.dp))
            LinearProgressIndicator(
                progress = { (glasses /
GOAL.toFloat()).coerceAtMost(1f) },
                modifier =
Modifier.fillMaxWidth().height(10.dp))
            Spacer(Modifier.height(16.dp))

Text("🥛".repeat(glasses.coerceAtMost(GOAL)),
                 fontSize = 28.sp)
            Row {
                Button(onClick = { scope.launch {
                    context.waterStore.edit {
                        it[GLASSES] = (it[GLASSES] ?:
0) + 1 }
                } }) { Text("+ 1 glass") }
                Spacer(Modifier.width(8.dp))
                OutlinedButton(onClick = {
scope.launch {
                    context.waterStore.edit {
it[GLASSES] = 0 }
                } }) { Text("Reset") }
            }
            Spacer(Modifier.height(24.dp))
            Card(Modifier.fillMaxWidth()) {
                Row(Modifier.padding(16.dp),
                    verticalAlignment =
Alignment.CenterVertically) {
                    Text("⏰ Hourly reminders",
Modifier.weight(1f))
                    Switch(checked = reminders,
onCheckedChange = { on ->
                        if (on &&
Build.VERSION.SDK_INT >= 33) {
                            permission.launch(
```

```
Manifest.permission.POST_NOTIFICATIONS)
                        } else saveReminders(on)
                    })
                }
            }
        }
    }
}
```

values/strings.xml

```
<plurals name="glasses_today">
    <item quantity="one">%d glass today</item>
    <item quantity="other">%d glasses today</item>
</plurals>
```

✍️ Try it yourself

1) Reset the counter automatically every new day.
2) Add an **“I drank 💧”** action button to the notification.
3) Let the user change the daily goal with a Slider saved in DataStore.

⚡ Android Platform Cheat Sheet

System APIs from Compose

Task	Code
Context	`LocalContext.current`
String	`stringResource(R.string.x, arg)`
Open URL	`Intent(ACTION_VIEW, Uri.parse(url))`
Share	`Intent.createChooser(sendIntent, null)`
Get result	`rememberLauncherForActivityResult(contract)`
Pick photo	`PickVisualMedia()`
Ask permission	`RequestPermission()`
Channel	`NotificationChannel(id, name, importance)`
Notify	`NotificationManagerCompat.from(c).notify()`
Background	`CoroutineWorker` + `WorkManager.enqueue()`
Foreground svc	`ServiceCompat.startForeground()`
Listen to events	`DisposableEffect { register; onDispose { } }`
Settings	`preferencesDataStore("name")`
Share a file	`FileProvider.getUriForFile()`
Listener → Flow	`callbackFlow { awaitClose { } }`
Survive death	`SavedStateHandle`, `rememberSaveable`
Back	`BackHandler(enabled) { }`

ANSWERS

Exercise Solutions

Water Reminder challenges



1) Reset every new day

DailyReset.kt

```
val DAY = stringPreferencesKey("day")

suspend fun Context.addGlass() = waterStore.edit {
    val today = LocalDate.now().toString()
    if (it[DAY] != today) {            // new day →
start from 0
        it[DAY] = today
        it[GLASSES] = 0
    }
    it[GLASSES] = (it[GLASSES] ?: 0) + 1
}

// Button(onClick = { scope.launch
{ context.addGlass() } })
```

2) “I drank” notification action

DrinkReceiver.kt

```
class DrinkReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent:
Intent) {
        val pending = goAsync()            // allow
async work
        CoroutineScope(Dispatchers.IO).launch {
            try {
                context.addGlass()

NotificationManagerCompat.from(context).cancel(1)
            } finally {
                pending.finish()
            }
        }
    }
}

// in ReminderWorker's builder:
val drink = PendingIntent.getBroadcast(ctx, 1,
    Intent(ctx, DrinkReceiver::class.java),
    PendingIntent.FLAG_IMMUTABLE)
// .addAction(R.drawable.ic_water, "I drank 💧",
drink)
```



📱 What you see

AndroidManifest.xml

```
<receiver android:name=".DrinkReceiver"
android:exported="false" />
```

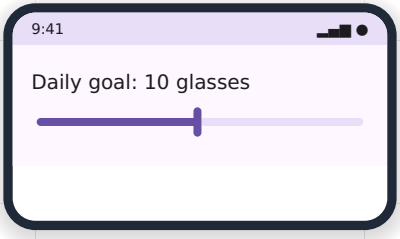
3) Goal slider

GoalSlider.kt

```
val GOAL_KEY = intPreferencesKey("goal")

val goal = prefs?.get(GOAL_KEY) ?: 8

Text("Daily goal: $goal glasses")
Slider(
    value = goal.toFloat(),
    onValueChange = { v -> scope.launch {
        context.waterStore.edit { it[GOAL_KEY] =
v.roundToInt() }
    } },
    valueRange = 4f..16f,
    steps = 11
)
```



📱 What you see

🎉 You now know the Android platform, not just the UI!
Next steps: build widgets with Glance, support Wear OS and tablets, add in-app purchases with Play Billing, and publish your first app on Google Play.